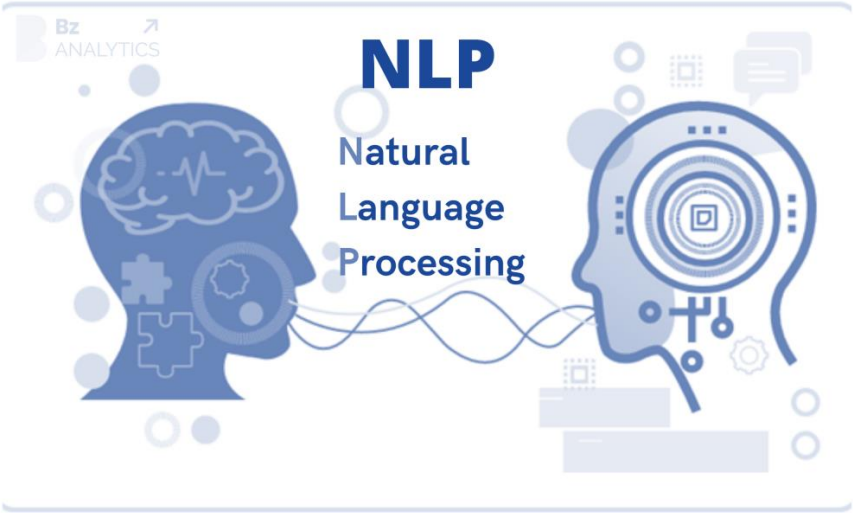
رواد مصر الرقمية

وزارة الاتصالات
وتكنولوجيا المعلومات



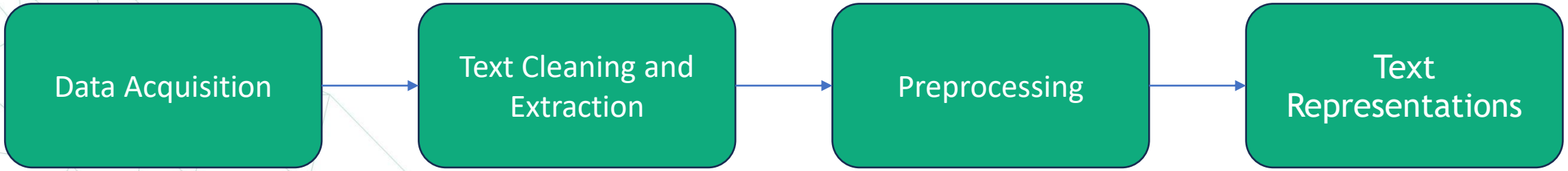
Natural language processing (NLP)

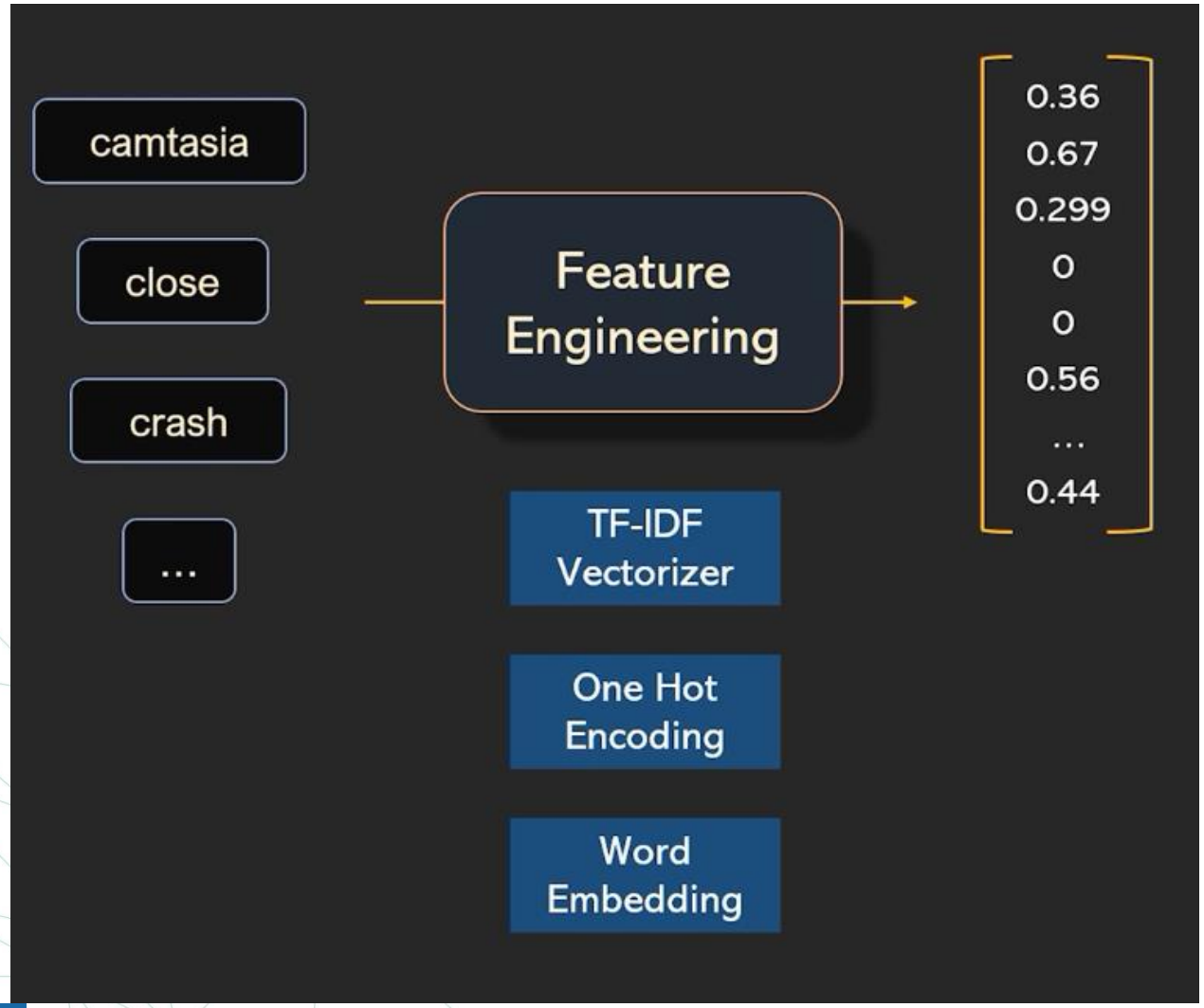
Session Two

Outline:

12	Bag of words
13	N-GRAM
14	TF-IDF
15	One Hot Encoding
16	Word Embedding(word2vec)

NLP Pipeline




camtasia
close
crash
...
Feature Engineering
TF-IDF Vectorizer
One Hot Encoding
Word Embedding
0.36
0.67
0.299
0
0
0.56
...
0.44

رواد مصر الرقمية

وزارة الاتصالات وتكنولوجيا المعلومات

Quick Recap :

Preprocessing done on the text before converting it into numbers (Text Representations)

- Tokenization: Breaking text into smaller parts, like words or sentences.
- Stemming and Lemmatization: Simplifying words to their base form.
- Regular Expression: A pattern used to search for and manipulate text.
- POS (Part-of-Speech): Identifying the grammatical category of a word (e.g., noun, verb).
- NER (Named Entity Recognition): Identifying and categorizing named entities in text (e.g., person names, locations).
- Spell Correction: Fixing spelling mistakes in text.

What are Bag of words ?

- It is a technique used to convert text into numeric numbers .
- It is used with important tasks in NLP like text classification and sentiment analysis.
- "Very similar to one hot encoding."
- All it does is take from my corpus, which contains all the words I have, the unique words, and then converts them to vectors. So, whenever you encounter this word anywhere, you remove it and put its vector in its place.

	the	red	dog	cat	eats	food
1. the red dog →	1	1	1	0	0	0
2. cat eats dog →	0	0	1	1	1	0
3. dog eats food →	0	0	1	0	1	1
4. red cat eats →	0	1	0	1	1	0

- The order of the words here is not important and doesn't affect the formation of the vector. What matters is the frequency of each word.

Document D1	*The child makes the dog happy* the: 2, dog: 1, makes: 1, child: 1, happy: 1
Document D2	*The dog makes the child happy* the: 2, child: 1, makes: 1, dog: 1, happy: 1

↓

	child	dog	happy	makes	the	**BoW Vector representations**
D1	1	1	1	1	2	**[1,1,1,1,2]**
D2	1	1	1	1	2	**[1,1,1,1,2]**

Notes:

- It's better to convert all your letters to small before applying the BoW because if it encounters the word "he" and "He", it will put each one of them in a separate column and calculate their frequency separately.

- Also, remove the stop words because they are very repetitive, so they will have high frequency numbers, and then the model will mistakenly think that they are more important, which is wrong.

Sentence 1 → He is a good boy
Sentence 2 → she is a good girl
Sentence 3 → boy and girl are good

→ Remove stop words

Sentence 1 → good boy
Sentence 2 → good girl
Sentence 3 → boy girl good

	good	boy	girl	Output/label
Sentence 1	1	1	0	
Sentence 2	1	0	1	
Sentence 3	1	1	1	

رواد مصر الرقمية

وزارة الاتصالات
وتكنولوجيا المعلومات

Bag-of-Words using sklearn

```
import pandas as pd
from sklearn.model_selection import train_test_split
from sklearn.feature_extraction.text import CountVectorizer
```

```
data = {
    'text': [
        'I love programming in Python',
        'Python is an amazing language',
        'I dislike bugs in the code',
        'Debugging is fun',
        'I enjoy learning new technologies',
        'This is a boring task',
        'I find machine learning fascinating',
        'I hate getting errors in my code'
    ],
    'label': ['positive', 'positive', 'negative', 'positive', 'positive', 'negative', 'positive', 'negative']
}
```

رواد مصر الرقمية

وزارة الاتصالات وتكنولوجيا المعلومات

Bag-of-Words using sklearn

```
# Convert to DataFrame
df = pd.DataFrame(data)

# Step 3: Train-Test Split
X_train, X_test, y_train, y_test = train_test_split(df['text'], df['label'], test_size=0.2, random_state=42)

# Step 4: Create Bag-of-Words Representation
vectorizer = CountVectorizer()
X_train_bow = vectorizer.fit_transform(X_train)
X_test_bow = vectorizer.transform(X_test)
```

```
# Convert the Bag-of-Words representation to a dense array
X_train_bow_dense = X_train_bow.toarray()

# Get the feature names (words) from the vectorizer
feature_names = vectorizer.get_feature_names_out()

# Create a DataFrame for better readability
bow_df = pd.DataFrame(X_train_bow_dense, columns=feature_names)

# Print the DataFrame
print(bow_df)

```

```
   bugs  code  debugging  dislike  enjoy  errors  fascinating  find  fun  \
0     0     0          0        0      0       0            0     0    0
1     0     1          0        0      0       1            0     0    0
2     1     1          0        1      0       0            0     0    0
3     0     0          0        0      1       0            0     0    0
4     0     0          1        0      0       0            0     0    1
5     0     0          0        0      0       0            1     1    0
```

What are the disadvantages of a bag of words?

Frequency is the measure :

- Frequency measures how many times each words appears in a sentence, with numbers like 0, 1, 2, 3 showing this. A word repeated often, like 4 times, is considered very important. However, a word that appears only once can still be important for the sentence's meaning, so it's given a 1. Other models, like TF-IDF, show how important each word is in a different way.

There is no semantic understanding:

- It happens that a lot of information is lost because it deals with words individually, meaning that all the words are independent, so the meaning is sometimes lost.

What are the disadvantages of a bag of words ?

Sparse representation :

You'll find that your matrix in the end has a lot of zeros if your documentation is large.

$$\begin{bmatrix} 0 & 1 & 0 & 0 \\ 0 & 0 & 0 & 0 \\ 0 & 5 & 0 & 2 \\ 9 & 0 & 0 & 6 \\ 7 & 0 & 0 & 0 \end{bmatrix}$$

N-GRAM:

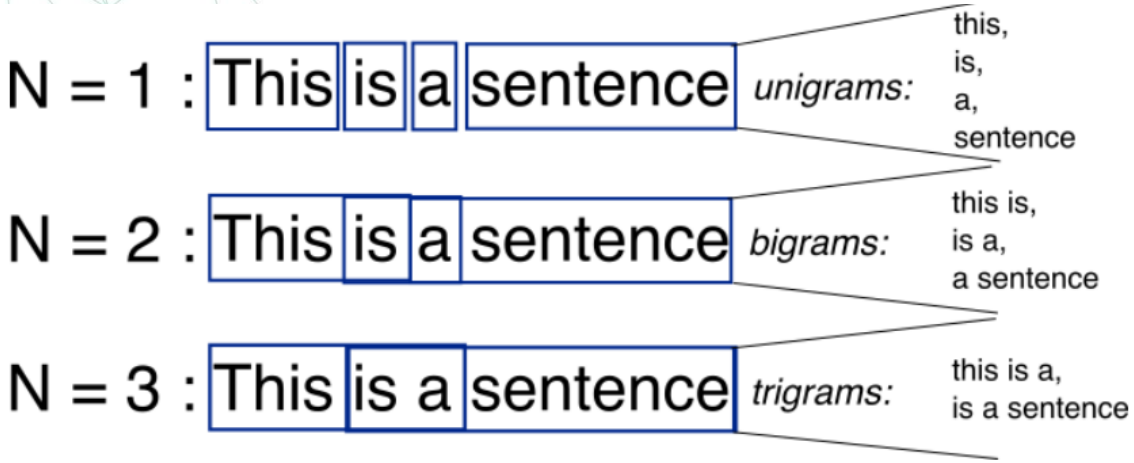
- Thinking of dealing with each word individually isn't the best because it makes me lose the sentence's meaning.

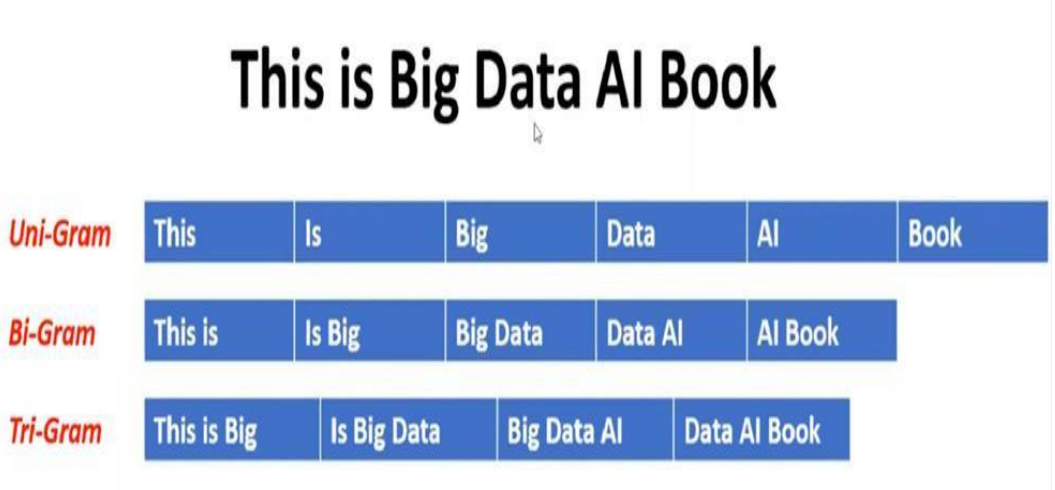
- For example, if we take each word separately, "book" could mean a book in both instances. However, when it's used with another word before or after it, the meaning changes. It could be understood as a verb, meaning to reserve, and the second instance would be understood as a book.

we need to book our tickets soon
we need to read this book soon

We have many types of N-GRAM :

I have many N-GRAMs, not just two (Bi-Gram).





رواد مصر الرقمية

وزارة الاتصالات وتكنولوجيا المعلومات

N-GRAM in sklearn :

```python
# Set ngram_range to (1, 2) for unigrams and bigrams
vectorizer = CountVectorizer(ngram_range=(1, 2))
X_train_bow = vectorizer.fit_transform(X_train)
X_test_bow = vectorizer.transform(X_test)

# Convert the Bag-of-Words representation to a dense array
X_train_bow_dense = X_train_bow.toarray()

# Get the feature names (unigrams and bigrams) from the vectorizer
feature_names = vectorizer.get_feature_names_out()

# Create a DataFrame for better readability
bow_df = pd.DataFrame(X_train_bow_dense, columns=feature_names)

# Print the DataFrame
display(bow_df)
```

```python
data = {
    'text': [
        'I love programming in Python',
        'Python is an amazing language',
        'I dislike bugs in the code',
    ],
    'label': ['positive', 'positive', 'negative']
}
```

	amazing	amazing language	an	an amazing	bugs	bugs in	code	dislike	dislike bugs	in	in the	is	is an	language	python	python is	the	the code
	1	1	1	1	0	0	0	0	0	0	0	1	1	1	1	1	0	0
	0	0	0	0	1	1	1	1	1	1	1	0	0	0	0	0	1	1

TF-IDF :

TF-IDF stands for Term Frequency - Inverse Document Frequency

- We previously discussed using BOW and N-GRAM to represent my text as vectors.

- It has a big advantage over the BOW in that it focuses more on the weight in the sentence, meaning that the most influential word in the sentence will have a higher value or number than the other words.

- The "bag of words" (BOW) method worked on the frequency, meaning that words that are repeated often have a higher value, which is not correct because we find that rare words have more impact on the meaning of the sentence. Also, stopwords like "the," "and," and "or" are not influential at all. So, I used to remove them before applying the BOW method. TF-IDF gives a lower value to frequently repeated words because they are not as influential as stopwords

How TF-IDF works :

- It calculates the TF for each **unique word**, which is the number of times each word appears in the
  sentence divided by the total number of words in that sentence.

- To get the IDF for each **unique word,** we calculate the **logarithm** of the total number of sentences I have divided by the number of sentences containing that word.

- After that, **multiply each column** in the TF by the IDF column to get the **final vectors.**

- In the end, I will have a **set of features** that are the **unique words** in the form of **vectors**.

Example

- Sen 1 : good boy
- Sen 2 : good girl
- Sen 3 : boy gilr good
- First we calculate TF → $TF = \frac{\text{No.of rep in a sen}}{\text{No.of words in a sen}}$
- $IDF = \log\left(\frac{\text{no. of } sentances}{\text{No.of sentences containing words}}\right)$

	sen1	sen2	sen3
Good	1/2	1/2	1/3
Boy	1/2	0	1/3
girl	0	1/2	1/3

X

words	IDF
Good	Log(3/3)=0
Boy	Log(3/2)
girl	Log(3/2)

	good	boy	girl	output
Sen 1	0	½*log(3/2)	0	
Sen 2	0	0	½*log(3/2)	
Sen 3	0	1/3*log(3/2)	1/3*log(3/2)	

رواد مصر الرقمية

وزارة الاتصالات
وتكنولوجيا المعلومات

TF-IDF in sklearn

```
vectorizer = TfidfVectorizer()
X_train_tfidf = vectorizer.fit_transform(X_train)
X_test_tfidf = vectorizer.transform(X_test)

# Convert the TF-IDF representation to a dense array
X_train_tfidf_dense = X_train_tfidf.toarray()

# Get the feature names (words) from the vectorizer
feature_names = vectorizer.get_feature_names_out()

# Create a DataFrame for better readability
tfidf_df = pd.DataFrame(X_train_tfidf_dense, columns=feature_names)

# Print the DataFrame
display(tfidf_df)
```

	bugs	code	debugging	dislike	enjoy	errors	fascinating	find	fun	getting	..
0	0.000000	0.000000	0.00000	0.000000	0.000000	0.000000	0.000000	0.000000	0.00000	0.000000	
1	0.000000	0.361281	0.00000	0.000000	0.000000	0.440579	0.000000	0.000000	0.00000	0.440579	
2	0.490779	0.402446	0.00000	0.490779	0.000000	0.000000	0.000000	0.000000	0.00000	0.000000	
3	0.000000	0.000000	0.00000	0.000000	0.521823	0.000000	0.000000	0.000000	0.00000	0.000000	
4	0.000000	0.000000	0.57735	0.000000	0.000000	0.000000	0.000000	0.000000	0.57735	0.000000	
5	0.000000	0.000000	0.00000	0.000000	0.000000	0.000000	0.521823	0.521823	0.00000	0.000000	

6 rows × 22 columns

One hot encoding

One-hot encoding is a technique used in natural language processing (NLP) to represent words as vectors in a high-dimensional space. This method is particularly useful for converting categorical text data into a numerical format

Hello, I'm Ironman. I have Friday AI

One Hot Encoding →

AI	1	0	0	0	0	0	0
Ironman	0	1	0	0	0	0	0
Friday	0	0	1	0	0	0	0
have	0	0	0	1	0	0	0
Hello	0	0	0	0	1	0	0
I	0	0	0	0	0	1	0
I'm	0	0	0	0	0	0	1

One hot encoding isn't good for representing data with large sizes why?

Let's assume that my vocabulary has 10k unique words. Using the one-hot encoding method, for example, when the word "man" is encountered, it will look at its index and set it to 1, while all other indices (from 0 to 9999) will be set to 0.

This approach creates three problems:

- It will show me a very large number of features.
- It causes a problem with the **sparse matrix**, meaning that most of the values are zeros and a very few number are ones."
- It won't preserve the semantic for me. Why? Because if a word, for example**, 'orange, apple,' are both very close to each other, but if one comes first and the other comes last, you will find that their vectors are completely different from each other."**

Word embeddings

- A word embedding is a learned representation for text where words that have the same meaning have a similar representation.
- It is this approach to representing words and documents that may be considered one of the key breakthroughs of deep learning on challenging natural language processing problems.

Why are word embeddings needed?

رواد مصر الرقمية

وزارة الاتصالات وتكنولوجيا المعلومات

Why are word embeddings needed?

Let us consider the two sentences – “You can scale your business.” and “You can grow your business.” These two sentences have the same meaning. If we consider a vocabulary considering these two sentences, it will constitute of these words: **{You, can, scale, grow, your, business}.**

A **one-hot encoding of these words** would create a vector of length 6. The encodings for each of the words would look like this:

You: [1,0,0,0,0,0], Can: [0,1,0,0,0,0], Scale: [0,0,1,0,0,0], Grow: [0,0,0,1,0,0],
Your: [0,0,0,0,1,0], Business: [0,0,0,0,0,1]

In a 6-dimensional space, each word would occupy one of the dimensions, meaning that none of these words has any similarity with each other – irrespective of their literal meanings.

Word2Vec, a word embedding methodology, solves this issue and enables similar words to have similar dimensions and, consequently, helps bring context.

Word Embedding Algorithms

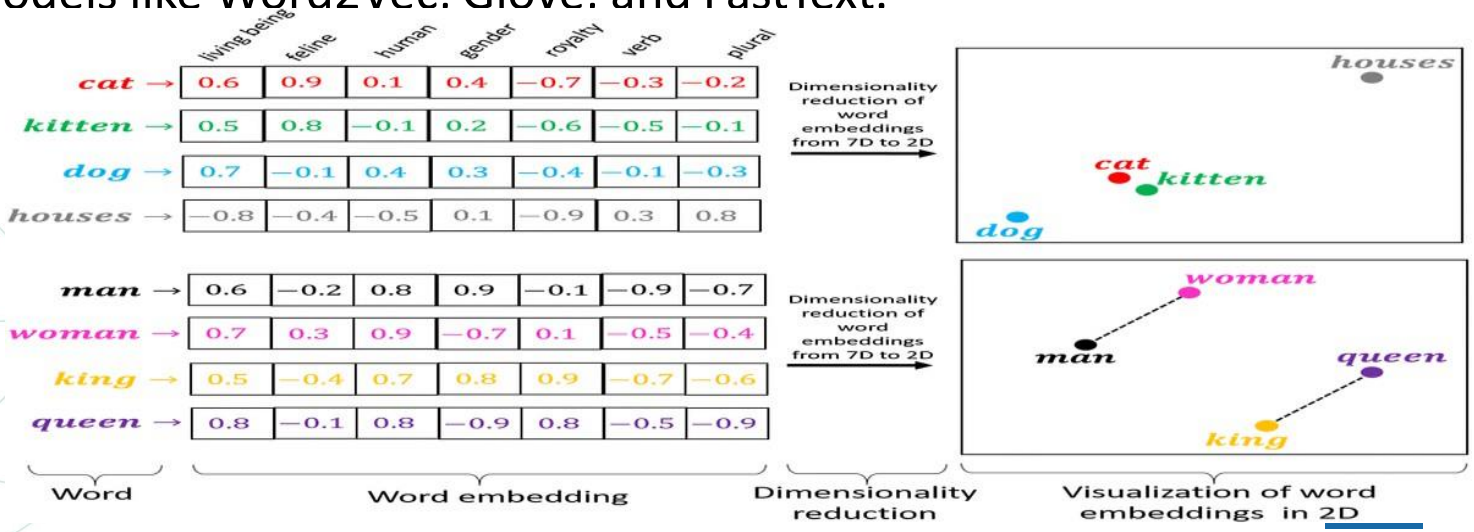
- **Embedding Layer**
- **Word2Vec**

Embedding Layer

- It's about algorithms I use to represent my words as vectors.

- But it's real advantage is that it can handle semantics, meaning words that have the same meaning or similar context. It gives them vectors that are close to each other, like apple, orange, and banana, which are all considered fruits. So you'll find that their vectors are close to each other, or like man and woman, and so on.

- I apply it using neural network models like Word2Vec. Glove. and FastText.



Man index 5k
Woman index 9k

man	woman
0	0
0	0
0	0
1	0
0	0
0	0
0	1

10k dim

Word embedding (feature representation)

	2000	5000	6000	9000	1000	7000
	Boy	Girl	king	queen	apple	mango
Gender	-1	1	-0.92	0.93	0.0	0.1
Royal	0.01	0.02	0.95	0.96	-0.02	0.01
Age	0.5	0.5	0.7	0.6	0.95	0.92
Food						
.						
.						

10 dim

Summary:

You define the vocab size and then get the indices of the words you have, which are the ones we will embedding to convert them into vectors, which are the features.

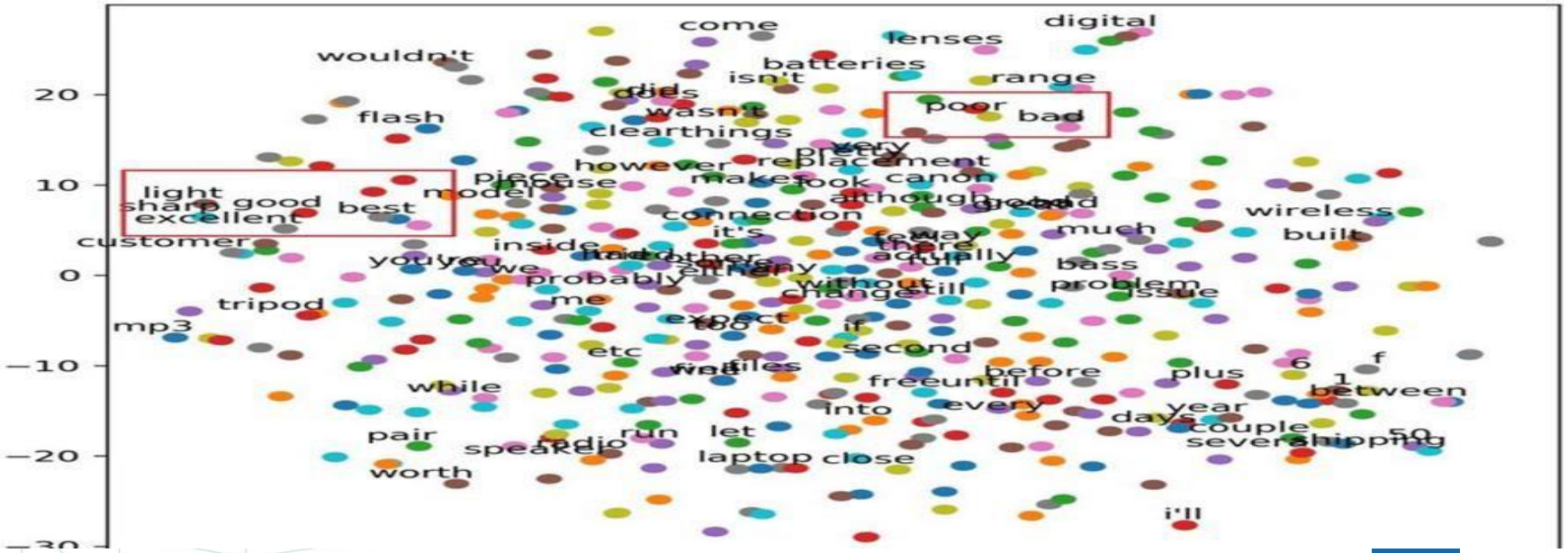
Also, we evaluate the word embedding through what???

Word similarity and word analogy

Notes:

1 The evidence that word embedding converts words into vectors while considering the semantic meaning, meaning that words with similar meanings will have close vectors.

2 In this image, the words that have undergone word embedding and then PCA are shown. If we look at words like best, good, excellent, we will find them very close to each other.



رواد مصر الرقمية

وزارة الاتصالات وتكنولوجيا المعلومات

Embedding Layer

```
from tensorflow.keras.preprocessing.text import one_hot
```

```
### sentences
sent=[  'the glass of milk',
        'the glass of juice',
        'the cup of tea',
       'I am a good boy',
        'I am a good developer',
        'understand the meaning of words',
        'your videos are good',]
```

```
### Vocabulary size>>>> the size of one hot encoding
voc_size=10000
#will make one hot encoding and return each sentence with it's indices
onehot_repr=[one_hot(words,voc_size)for words in sent]
print(onehot_repr)
```

```
[5853, 812, 6103, 5337], [5853, 812, 6103, 2686], [5853, 9090, 6103, 8774], [6067, 1010, 1760, 4399, 9258], [6067, 1010, 1760, 4399, 4352], [2859, 5853, 2496, 6103, 8076], [4424, 1094, 1493, 4399]]
```

رواد مصر الرقمية

وزارة الاتصالات وتكنولوجيا المعلومات

Embedding Layer

```
from tensorflow.keras.preprocessing.sequence import pad_sequences
sent_length=8
embedded_docs=pad_sequences(onehot_repr,padding='pre',maxlen=sent_length)
print(embedded_docs)
```

```
[   0    0    0    0 2304 4048  200 7085]
[   0    0    0    0 2304 4048  200 6410]
[   0    0    0    0 2304 1861  200 5028]
[   0    0    0 3536 9320 8210  995 1424]
[   0    0    0 3536 9320 8210  995 8627]
[   0    0    0 5261 2304  987  200 1845]
[   0    0    0    0 6496 1088 9507  995]]
```

```
from keras.models import Sequential
from keras.layers import Embedding
model = Sequential()
model.add(Embedding(voc_size, 10))
model.compile('adam', 'mse')
```

voc_size: The size of the vocabulary, i.e., the number of unique tokens or words in your dataset.
10: The size of the dense embedding vectors. Each token in your vocabulary will be mapped to a 10-dimensional vector.
This layer converts integer-encoded tokens into dense vectors of size 10.

رواد مصر الرقمية

وزارة الاتصالات وتكنولوجيا المعلومات

Embedding Layer

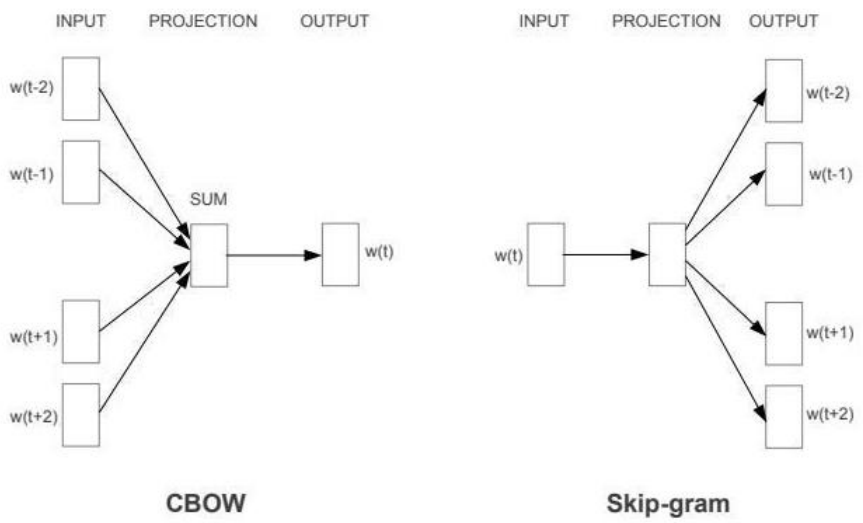
```
print(model.predict(embedded_docs))
```

```
#frist sentence after onehot + padding
embedded_docs[0]
```

```
array([   0,    0,    0,    0, 5853,  812, 6103, 5337], dtype=int32)
```

```
#frist sentence after embedding
print(model.predict(embedded_docs)[0])
```

```
1/1 ━━━━━━━━━━━━━━━━━━━━ 0s 20ms/step
[[ 0.02706667  0.04142978  0.03326375 -0.01022017  0.03536961  0.01260928
  -0.03661736 -0.00810615 -0.02808102 -0.0149346 ]
 [ 0.02706667  0.04142978  0.03326375 -0.01022017  0.03536961  0.01260928
  -0.03661736 -0.00810615 -0.02808102 -0.0149346 ]
 [ 0.02706667  0.04142978  0.03326375 -0.01022017  0.03536961  0.01260928
  -0.03661736 -0.00810615 -0.02808102 -0.0149346 ]
 [ 0.02706667  0.04142978  0.03326375 -0.01022017  0.03536961  0.01260928
  -0.03661736 -0.00810615 -0.02808102 -0.0149346 ]
 [ 0.00724664  0.03642967 -0.04656214 -0.02299914 -0.04470244  0.046987
   0.01280831 -0.03085253 -0.00669528 -0.04234318]
 [-0.00918596 -0.03141656 -0.03879803 -0.00557727  0.04712559  0.00407242
  -0.00995084 -0.01183487  0.02293731  0.00454862]
 [-0.00931799  0.01246104  0.00345091  0.00932175 -0.01391529  0.02653677
  -0.02076416 -0.01568348  0.00471094  0.0422902 ]
 [-0.00304006  0.04683647 -0.00914602 -0.0212866  -0.02520748 -0.00275395
  -0.00500282  0.02024866 -0.04726223  0.03458563]]
```



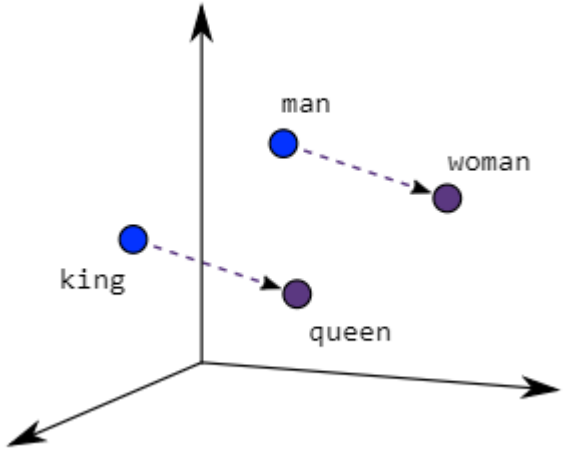
Word2vec

Word2Vec is a statistical method for efficiently learning a standalone word embedding from a text corpus.

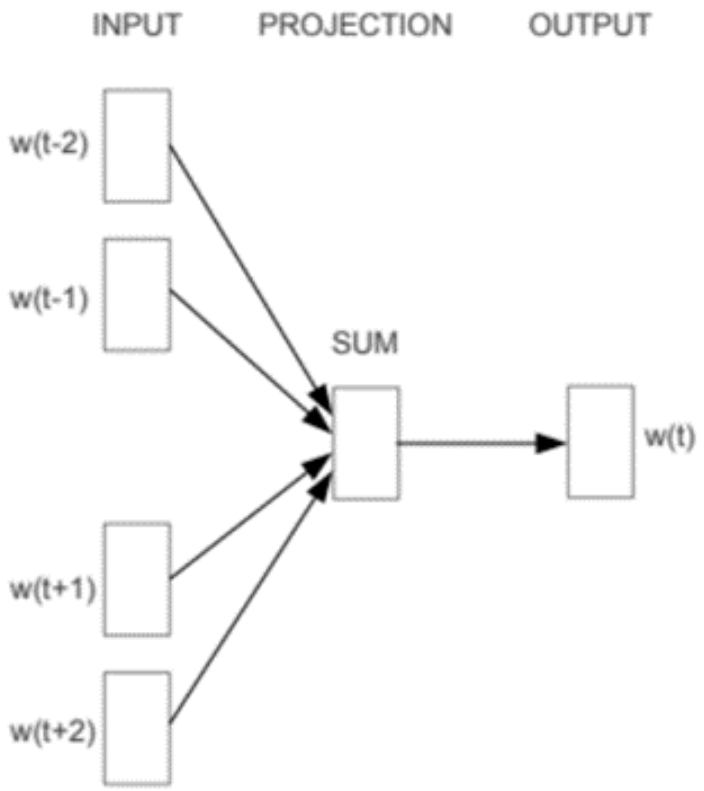
Two different learning models were introduced that can be used as part of the word2vec approach to learn the word embedding; they are:

Continuous Bag-of-Words, or CBOW model.

Continuous Skip-Gram Model.

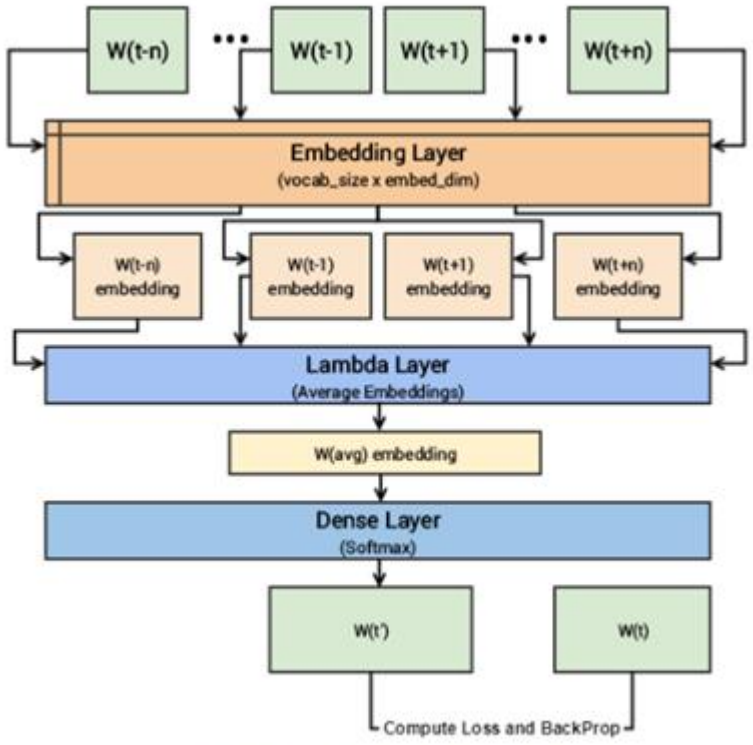


How does CBOW work?



- Even though Word2Vec is an unsupervised model where you can give a corpus without any label information and the model can create dense word embeddings, Word2Vec internally leverages a supervised classification model to get these embeddings from the corpus.
- The CBOW architecture comprises a deep learning classification model in which we take in context words as input, X, and try to predict our target word, Y.
- For example, if we consider the sentence – "Word2Vec has a deep learning model working in the backend.", there can be pairs of context words and target (center) words. If we consider a context window size of 2, we will have pairs like ([deep, model], learning), ([model, in], working), ([a, learning), deep) etc. The deep learning model would try to predict these target words based on the context words.

How does CBOW work?

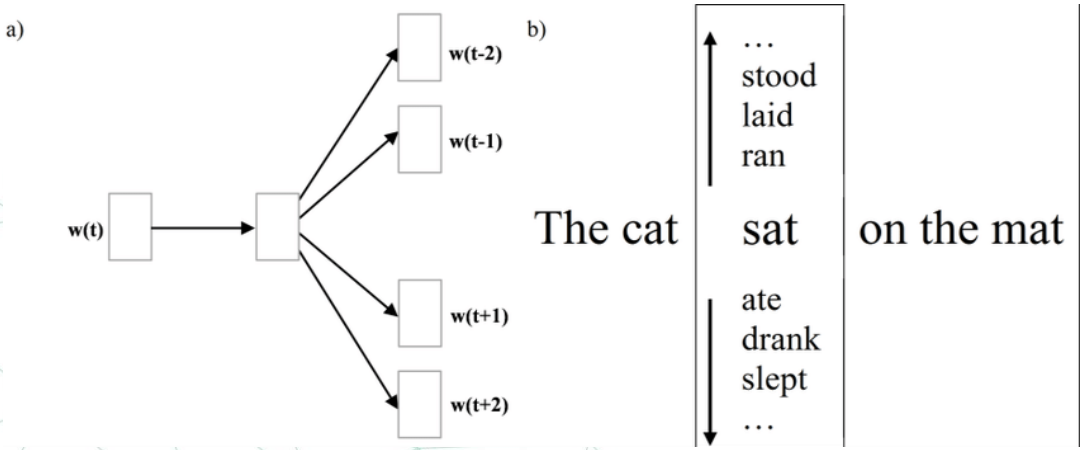


Visual depiction of the CBOW deep learning model

The following steps describe how the model works:

1. The context words are first passed as an input to an embedding layer (initialized with some random weights) as shown in the Figure below.
2. The word embeddings are then passed to a lambda layer where we average out the word embeddings.
3. We then pass these embeddings to a dense SoftMax layer that predicts our target word. We match this with our target word and compute the loss and then we perform backpropagation with each epoch to update the embedding layer in the process.
4. We can extract out the embeddings of the needed words from our embedding layer once the training is completed.

How does Skip-gram work?



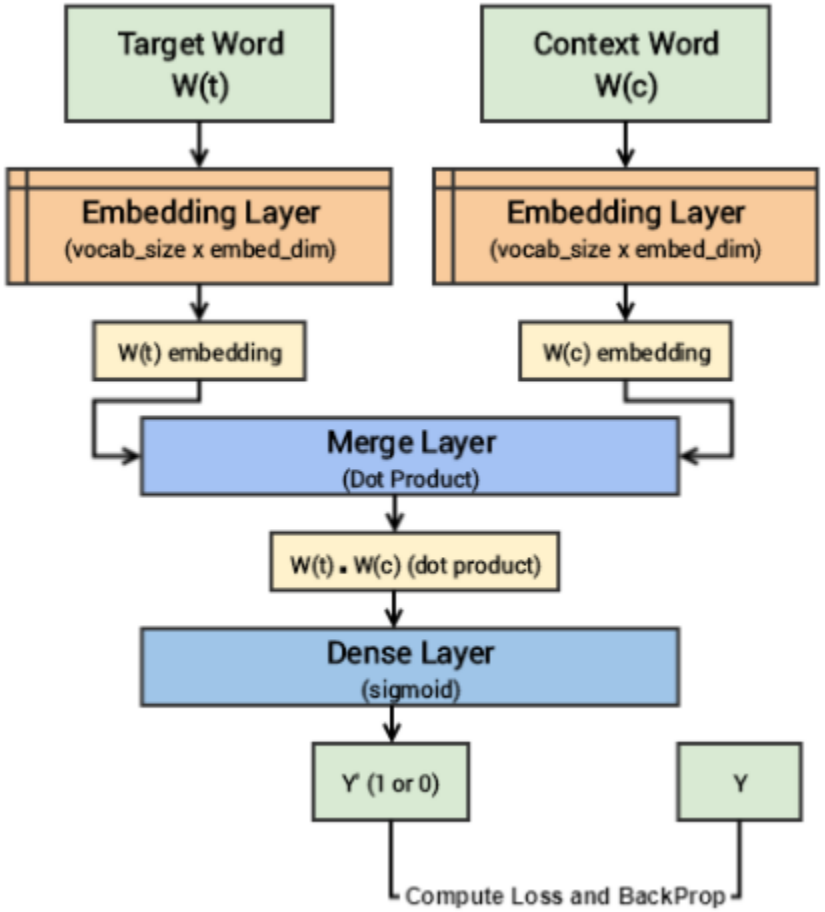
In the skip-gram model, given a target (centre) word, the context words are predicted using **word representations**. So, considering the same sentence – “Word2Vec has a **neural networks** working in the backend.” and a context window size of 2, given the center word ‘learning’, the model tries to predict [‘deep’, ‘model’] and so on. This prediction process is fundamental to **language modeling** and the creation of **vector representations** of words.

Since the skip-gram model has to predict multiple words from a single given word, we feed the model pairs of (X, Y) where X is our input and Y is our label. This is done by creating **positive input samples and negative input samples.**

How does Skip-gram work?

- Positive Input Samples will have the training data in this form: [(target, context),1] where the target is the target or center word, context represents the surrounding context words, and label 1 indicates if it is a relevant pair. Negative Input Samples will have the training data in the same form: [(target, random),0]. In this case, instead of the actual surrounding words, randomly selected words are fed in along with the target words with a label of 0 indicating that it's an irrelevant pair.
- These samples make the model aware of the contextually relevant words and consequently generate similar embeddings for similar meaning words.

How does Skip-gram work?



The following steps describe how the model works:

1. Both the target and context word pairs are passed to individual embedding layers from which we get dense word embeddings for each of these two words.
2. We then use a 'merge layer' to compute the dot product of these two embeddings and get the dot product value.
3. This dot product value is then sent to a dense sigmoid layer that outputs either 0 or 1.
4. The output is compared with the actual label and the loss is computed followed by backpropagation with each epoch to update the embedding layer in the process.

رواد مصر الرقمية

وزارة الاتصالات وتكنولوجيا المعلومات

Word2Vec in Python

```python
import gensim
from gensim.models import Word2Vec

# Sample data: list of sentences
sentences = [
    ['hello', 'world'],
    ['I', 'love', 'natural', 'language', 'processing','Like','pyhton'],
    ['word2vec', 'is', 'a', 'great', 'tool'],
    ['chatgpt', 'helps', 'with', 'coding', 'and', 'writing'],
    ['this', 'is', 'a', 'simple', 'example']
]
```

```python
# Train the Word2Vec model
model = Word2Vec(sentences, vector_size=50, window=3, min_count=1, sg=0)

# vector_size: Dimensionality of the word vectors.
# window: Maximum distance between the current and predicted word within a sentence.
# min_count: Ignores all words with total frequency lower than this.
# sg: Training algorithm: 1 for skip-gram; 0 for CBOW.

```

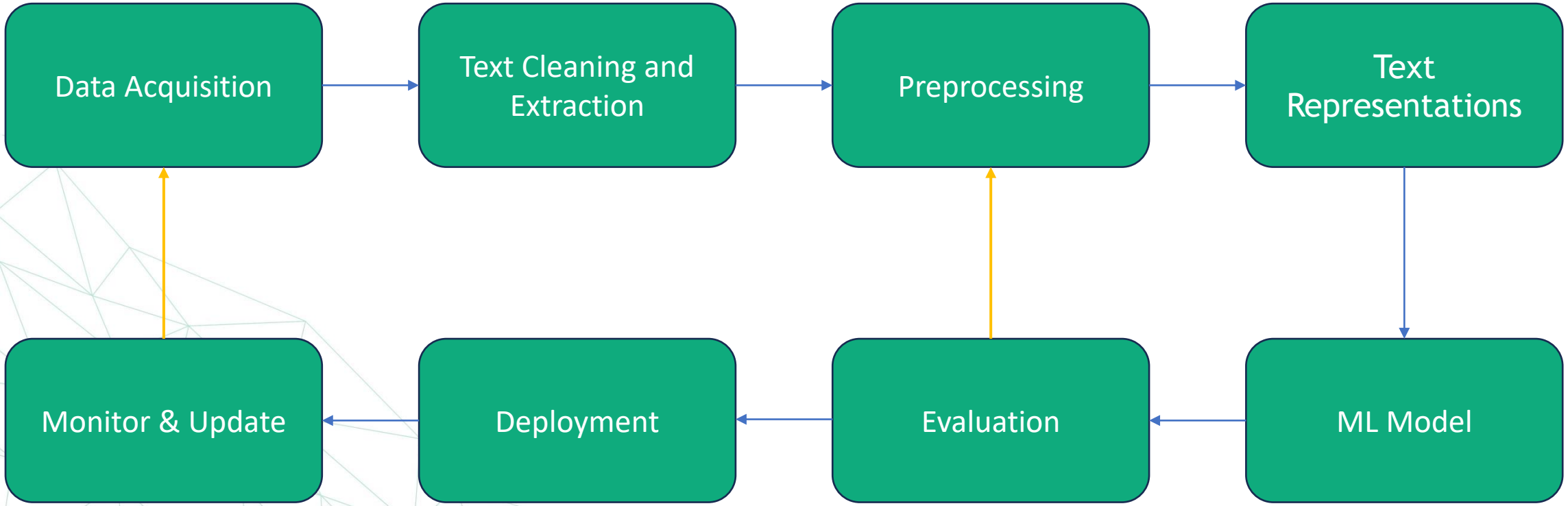
```python
# Get the vector for a specific word
word_vector = model.wv['word2vec']
print(word_vector)

```

رواد مصر الرقمية

وزارة الاتصالات وتكنولوجيا المعلومات

NLP Pipeline



رواد مصر الرقمية

وزارة الاتصالات
وتكنولوجيا المعلومات

Thanks